

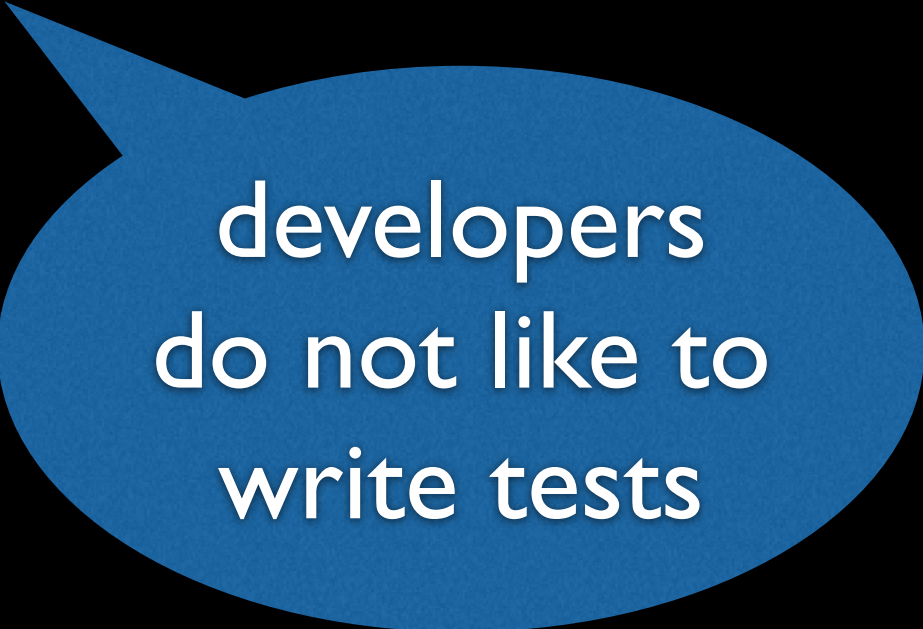
Merge and Code Review

Paulo Borba
Informatics Center
Federal University of Pernambuco

pauloborba.cin.ufpe.br

Test generation

Wouldn't it be good to
eliminate the test
creation cost?

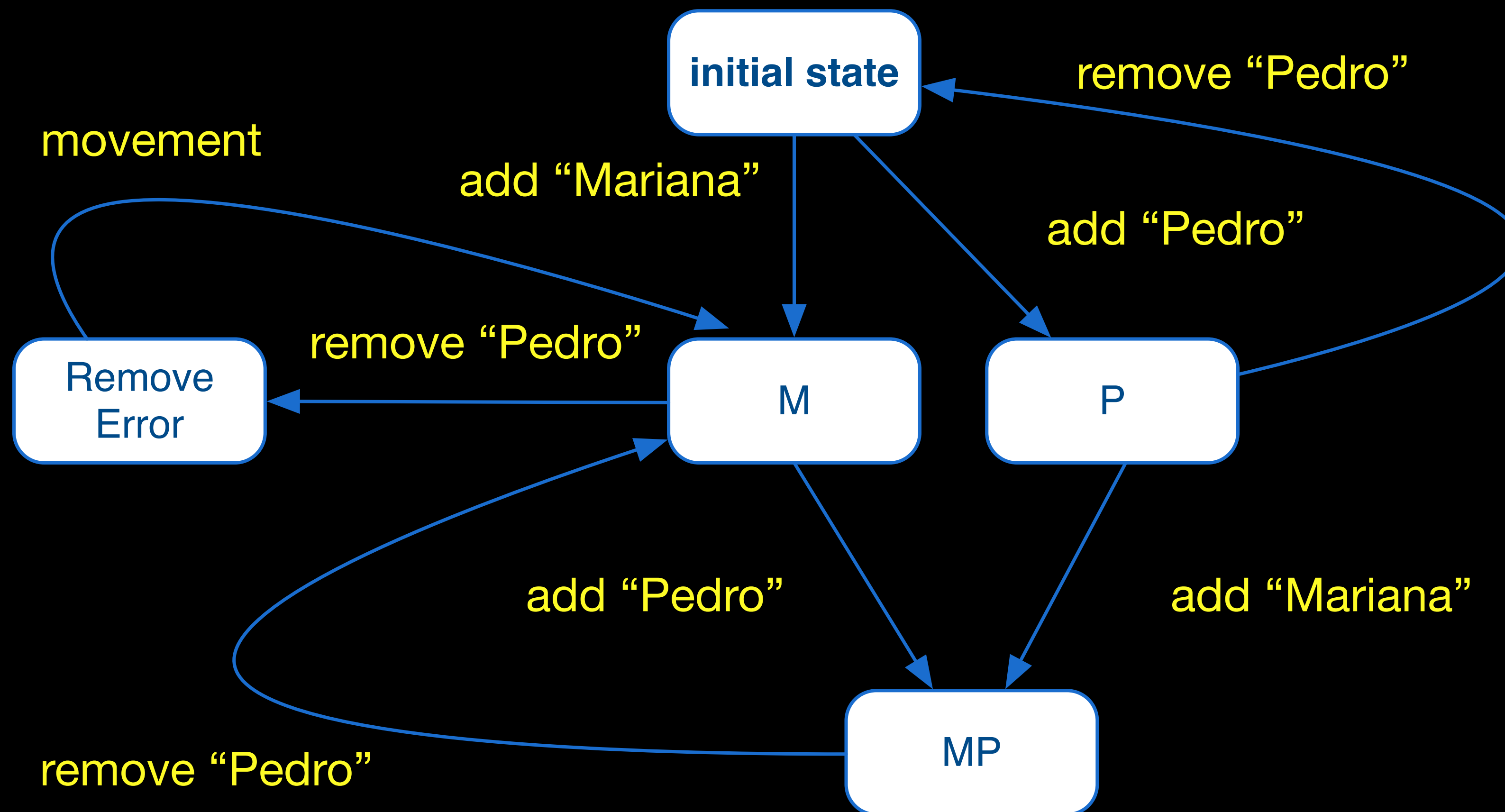


developers
do not like to
write tests

Semantic conflict detection
might benefit from that!

If we have system
specifications, formal
models, then we solve
the problem by...

generating paths in a graph!



Such approaches are
called model-based
testing

A particular case is
property-based testing

class
under test

```
class ExamGrades {  
    double[] grades;  
    ...  
    //@ requires grades != null  
    //@ ensures ascendingOrder(grades)  
    void sort() {  
        ...  
    }  
}
```

pre-condition

post-condition

tool generates inputs that
satisfy the pre-condition, runs the
method, and then check whether
the post-condition is valid

```
class ExamGrades {  
    double[] grades;  
    ...  
    //@ requires grades != null  
    //@ ensures ascendingOrder(grades)  
    void sort() {  
        ...  
    }  
}
```

```
a = new double[5];  
e = new Exam(a);  
e.sort();  
assert ascendingOrder(e.grades);
```

```
a = new double[] { 9, 5, 7, 0 };  
e = new Exam(a);  
e.sort();  
assert ascendingOrder(e.grades);
```

```
a = new double[] { 1, -5, 7, 11 };  
e = new Exam(a);  
e.sort();  
assert ascendingOrder(e.grades);
```

If we have no
specification other than
system code, we could
automatically
generate...

it("doesn't accept students with the same CPF ", () => {

var student: Student = new Student();

student.name = "Mariana";

student.cpf = "683";

class.add(student);

object
creation and
initialisation, test
setup,
input

student = new Student();

student.name = "Pedro";

student.cpf = "683";

class.add(student);

method calls,
test actions

expect(class.getStudents().length).toBe(1);

)

But how to guess the
assertions, expected
results?

it("doesn't accept students with the same CPF ", () => {

var student: Student = new Student();

student.name = "Mariana";

student.cpf = "683";

class.add(student);

student = new Student();

student.name = "Pedro";

student.cpf = "683";

class.add(student);

expect(class.getStudents().length).toBe(1);

})

system and test
case specific oracle

cannot be
automatically generated

The **oracle** problem can
be solved only when...

Function inverses are
available, or for...

generally applicable cases

Regression oracle: checks whether system behaves as previous version

System crashes oracle: checks whether execution raises an unexpected unhandled exception

System resource usage oracle: checks whether execution uses excessive resources

Random test generation
(regression oracle)

```
Object[] a = new Object[];  
LinkedList ll = new LinkedList();  
ll.addFirst(a);  
TreeSet ts = new TreeSet(ll);  
Set u = Collections.unmodifiableSet(ts);  
  
assert u.equals(u);
```

} input

} oracle

Assertion fails:
bug in JDK!

Randoop tests

```
public void test502() throws Throwable {
    if (debug)
        System.out.format("%n%s%n", "RegressionTest1.test502");
    org.antlr.v4.codegen.CodeGenerator codeGenerator0 = null;
    org.antlr.v4.codegen.target.Python2Target python2Target1 = new org.antlr.v4.codegen.target.Python2Target(codeGenerator0);
    java.lang.String str4 = python2Target1.getTargetStringLiteralFromString("hi!", true);
    java.lang.String str6 = python2Target1.getListLabel("");
    java.lang.String str8 = python2Target1.getImplicitSetLabel("\"4.5.2.1\"");
    boolean boolean9 = python2Target1.templatesExist();
    boolean boolean10 = python2Target1.templatesExist();
    java.lang.String str12 = python2Target1.getAltLabelContextStructName("_tset");
    java.util.Set<java.lang.String> strSet13 = python2Target1.getBadWords();
    java.lang.String str15 = python2Target1.getTargetStringLiteralFromString("tset\\\\"4.5.2.1");
    java.lang.String str17 = python2Target1.getImplicitRuleLabel("hi!");
    boolean boolean18 = python2Target1.supportsOverloadedMethods();
    org.junit.Assert.assertTrue("'" + str4 + "' != '" + "\"hi!\"" + "'", str4.equals("\"hi!\""));
    org.junit.Assert.assertTrue("'" + str6 + "' != '" + "" + "'", str6.equals(""));
    org.junit.Assert.assertTrue("'" + str8 + "' != '" + "_tset\"4.5.2.1\"" + "'", str8.equals("_tset\"4.5.2.1\""));
    org.junit.Assert.assertTrue("'" + boolean9 + "' != '" + true + "'", boolean9 == true);
    org.junit.Assert.assertTrue("'" + boolean10 + "' != '" + true + "'", boolean10 == true);
    org.junit.Assert.assertTrue("'" + str12 + "' != '" + "_tsetContext" + "'", str12.equals("_tsetContext"));
    org.junit.Assert.assertNotNull(strSet13);
    org.junit.Assert.assertTrue("'" + str15 + "' != '" + "\"tset\\\\\\\\\\\\\\\\\"4.5.2.1\"" + "'", str15.equals("\"tset\\\\\\\\\\\\\\\\\"4.5.2.1\""));
    org.junit.Assert.assertTrue("'" + str17 + "' != '" + "_hi!" + "'", str17.equals("_hi!"));
    org.junit.Assert.assertTrue("'" + boolean18 + "' != '" + false + "'", boolean18 == false);
}
```


GenerateSequences(*classes, contracts, filters, timeLimit*)

```
1  errorSeqs  $\leftarrow \{\}$  // Their execution violates a contract.
2  nonErrorSeqs  $\leftarrow \{\}$  // Their execution violates no contract.
3  while timeLimit not reached do
4    // Create new sequence.
5     $m(T_1 \dots T_k) \leftarrow \text{randomPublicMethod}(\text{classes})$ 
6     $\langle \text{seqs}, \text{vals} \rangle \leftarrow \text{randomSeqsAndVals}(\text{nonErrorSeqs}, T_1 \dots T_k)$ 
7    newSeq  $\leftarrow \text{extend}(m, \text{seqs}, \text{vals})$ 
8    // Discard duplicates.
9    if newSeq  $\in \text{nonErrorSeqs} \cup \text{errorSeqs}$  then
10      continue
11    end if
12    // Execute new sequence and check contracts.
13     $\langle \vec{\mathbf{o}}, \text{violated} \rangle \leftarrow \text{execute}(\text{newSeq}, \text{contracts})$ 
14    // Classify new sequence and outputs.
15    if violated = true then
16      errorSeqs  $\leftarrow \text{errorSeqs} \cup \{\text{newSeq}\}$ 
17    else
18      nonErrorSeqs  $\leftarrow \text{nonErrorSeqs} \cup \{\text{newSeq}\}$ 
19      setExtensibleFlags(newSeq, filters,  $\vec{\mathbf{o}}$ ) // Apply filters.
20    end if
21  end while
22  return  $\langle \text{nonErrorSeqs}, \text{errorSeqs} \rangle$ 
```

Randoop **class** tests

Input + Action: sequence of constructor and method calls of the class under test

Oracle: sequence of assert statements

Randomly generated

- Input + Action is executed to filter out invalid (contract breaking) and redundant sequences

Evolutionary test
generation
(regression oracle)

Evosuite tests

```
public void test05() throws Throwable {
    CommonToken commonToken0 = (CommonToken)Token.SKIP_TOKEN;
    AltAST altAST0 = new AltAST(commonToken0);
    Python2Target python2Target0 = new Python2Target((CodeGenerator) null);
    boolean boolean0 = python2Target0.visibleGrammarSymbolCausesIssueInGeneratedCode(altAST0);
    assertFalse(boolean0);
}

@Test(timeout = 4000)
public void test06() throws Throwable {
    Python2Target python2Target0 = new Python2Target((CodeGenerator) null);
    STGroup stGroup0 = python2Target0.getTemplates();
    assertFalse(stGroup0.iterateAcrossValues);
}
```

input and
actions

oracle

Evosuite genetic algorithm

Population of test suites (set of test cases as in Randoop)

Fitness function (test suite coverage, etc.) selects fittest test suites (individuals) for reproduction

Crossover randomly exchange test cases between test suites

Mutation might add or change (arguments, method calls) test cases

Similar tools, with focus
on system crash and
resource usage oracles,
are generally called
fuzzers

Start from valid inputs,
change them and add to
the poll if coverage is
increased, further
change inputs in the
pool...

Generation gives better results when code...

- has assertions, which help as oracles
- has no hidden inputs (files, OS state, etc.), which lead to nondeterminism
- can work with stubs for interposing on I/O and state
- avoids fuzzer blockers (strong validity properties on test input)
- is fast (or can turn off irrelevant slow parts)

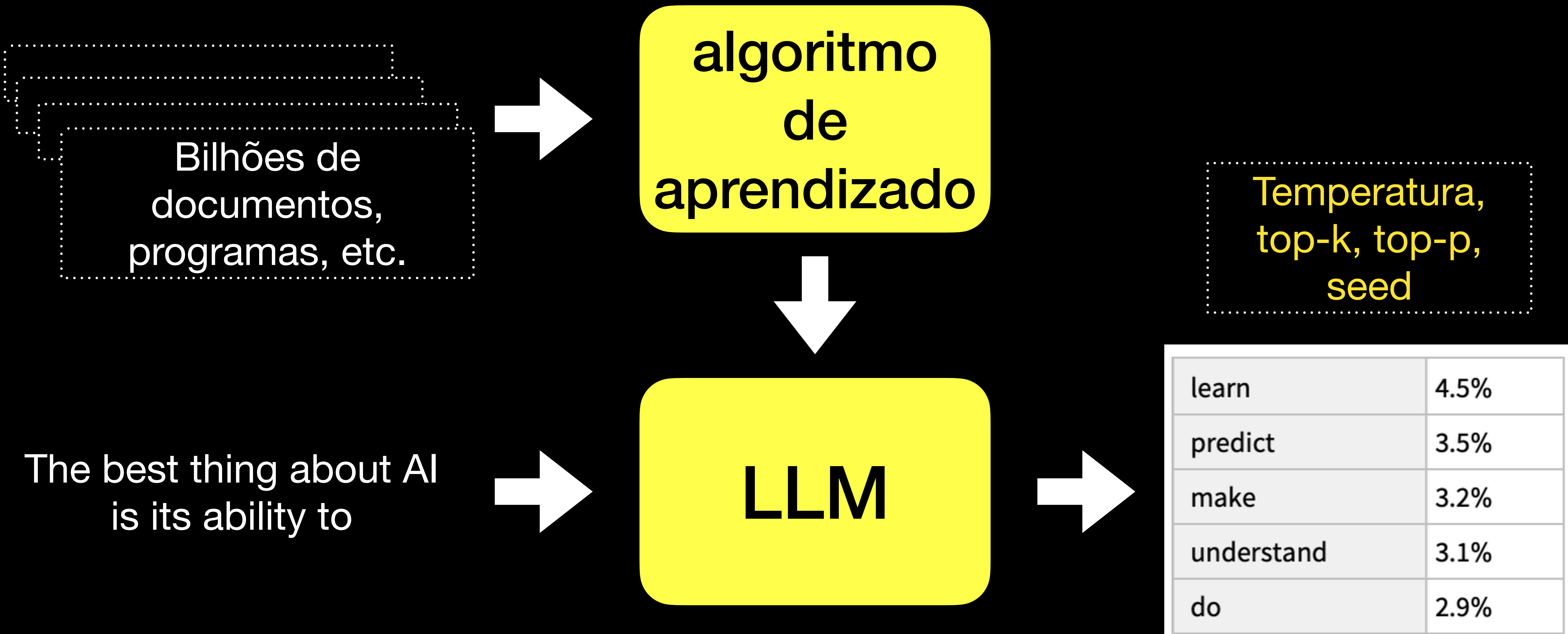
LLMs and Agents

LLM

simulação grosseira de um
sábio com problemas
cognitivos

Software Is Changing (Again), Andrej Karpathy
<https://www.youtube.com/watch?v=LCEmiRjPEtQ>

Aleatoriedade na escolha da próxima palavra



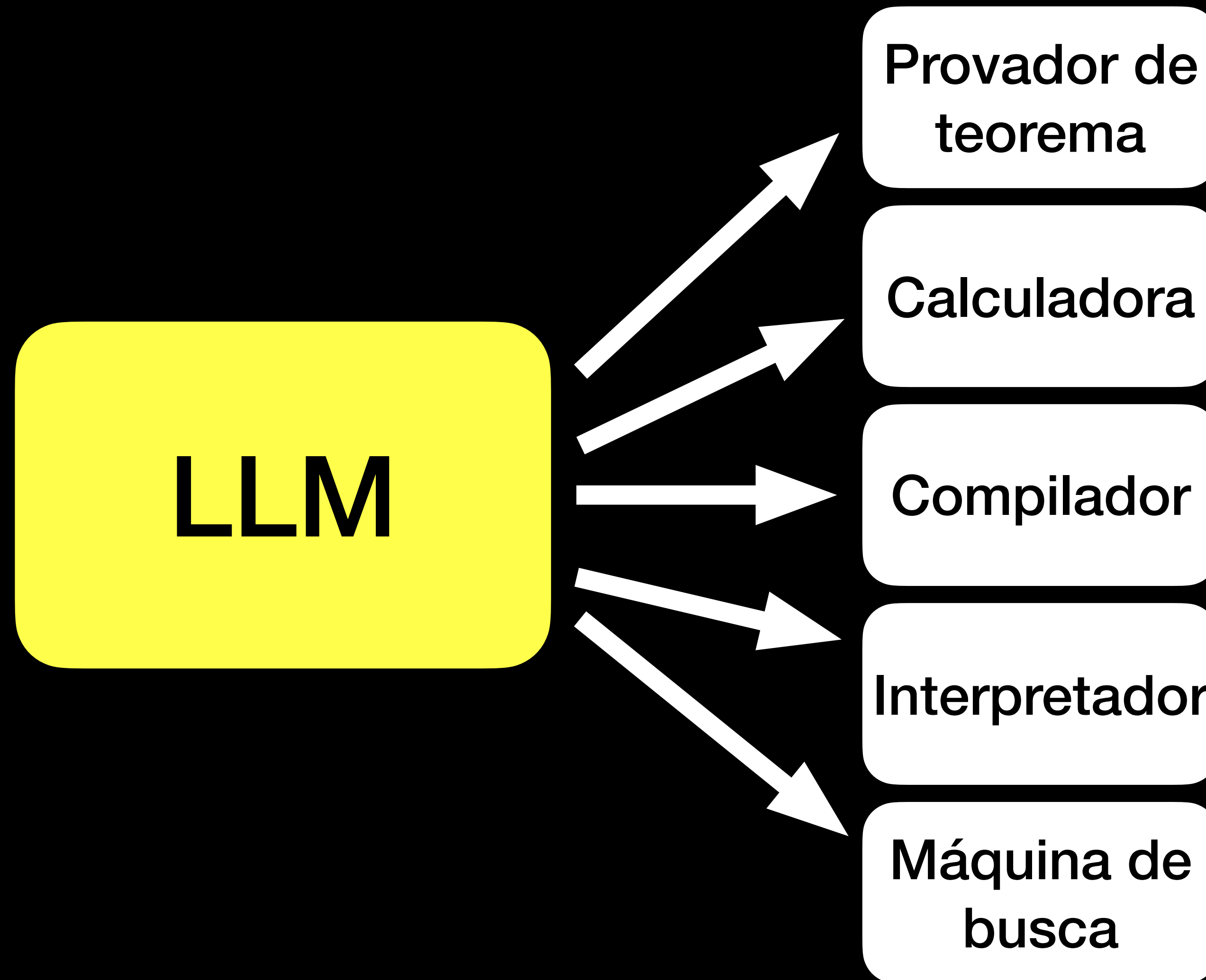
Consequências

- Impressionante capacidade e conhecimento
- Impressionante incapacidade e desconhecimento
- Alucinações
- Curta memória, limite de atenção
- Contexto local

Agente (IA)

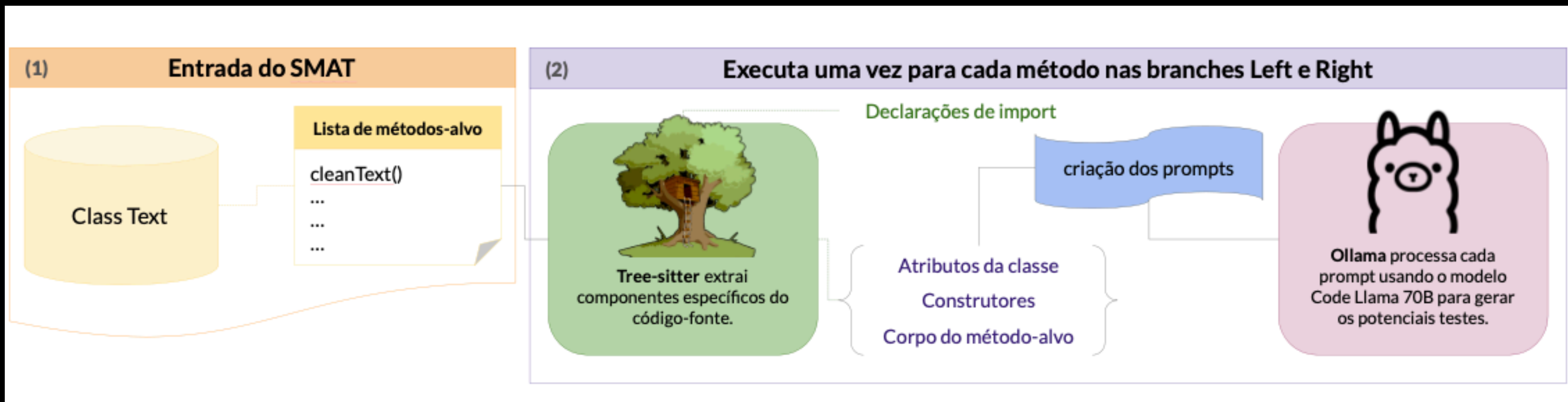
trabalha com LLMs para tentar
identificar e corrigir suas
limitações

Augmented LLMs



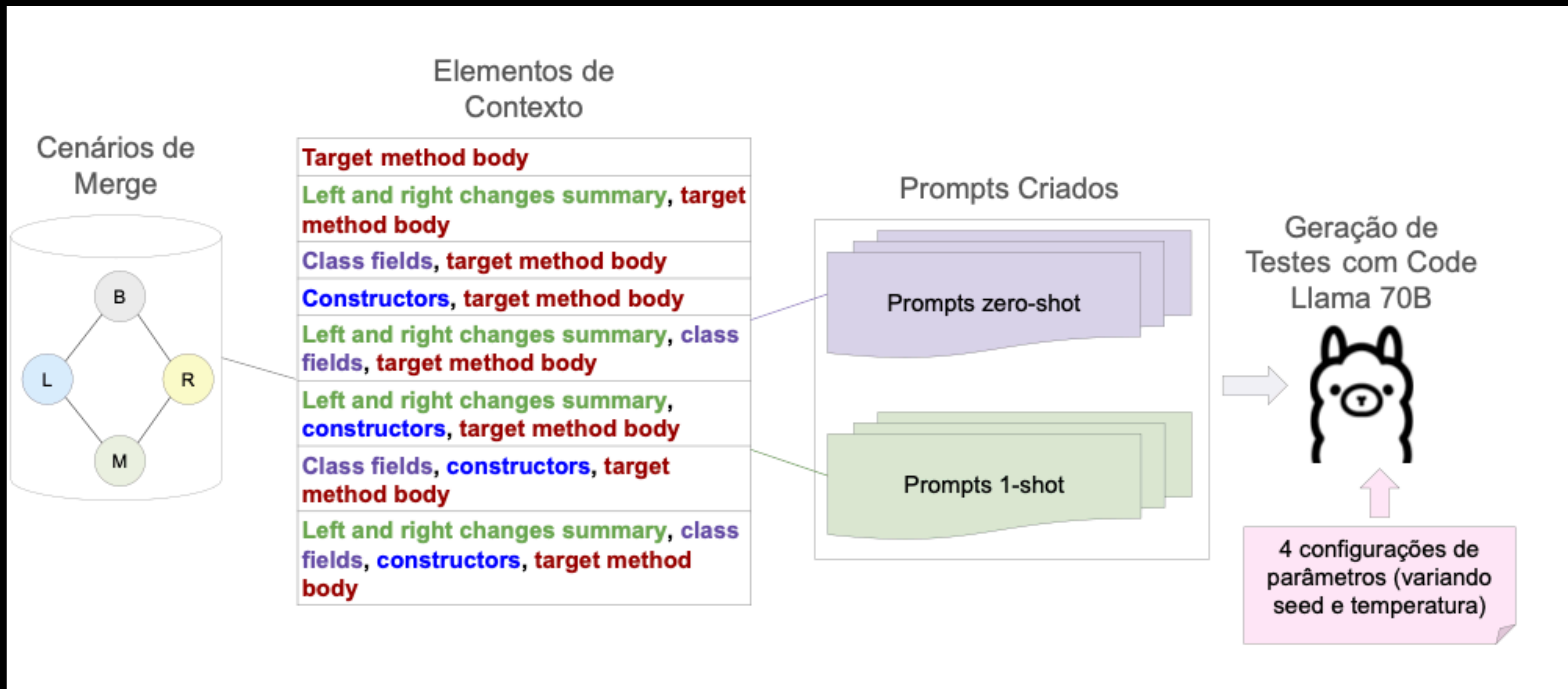
Detecção de conflitos semânticos de código

geração de testes de unidade com LLMs



<https://arxiv.org/abs/2507.06762>

Analizando várias soluções



Resultados inferiores a outras ferramentas com maior (~20x) custo computacional

Ferramenta de Geração de Testes	Conflitos Detectados
Differential EvoSuite	6
EvoSuite	5 [1]
Code Llama 70B (união de todas as execuções)	5 (1*)
Code Llama 70B (zero-shot, temperatura 0)	3 (1*)
Randoop	2
Randoop Clean	2 [1]

**Substancial diferença de
comportamento dependendo do
sistema (amostra)**

**Com sistemas simples, aumento de
200-600% na taxa de testes gerados
que compilam**

Test generation

Merge and Code Review

Paulo Borba
Informatics Center
Federal University of Pernambuco

pauloborba.cin.ufpe.br